МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

ОТЧЁТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ
Проектная практика

приказ Университета о направлении на практику от «10» февраля 2026 г. № 1023-С

Отчет представлен к рассмотрению:
Студент группы ИКБО-12-22 «21» марта 2026 г. Щербак М.А.
(подпись и расшифровка подписи)

Отчет утвержден.
Допущен к защите:

Руководитель практики от кафедры «21» марта 2026 г. Бирюкова А.А.
(подпись и расшифровка подписи)

Москва 2026 г.

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий (МОСИТ)

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА ПРОИЗВОДСТВЕННУЮ ПРАКТИКУ
Проектная практика

Студенту 4 курса учебной группы ИКБО-12-22
Щербаку Максиму Александровичу

Место и время практики: РТУ МИРЭА кафедра МОСИТ, с 09 февраля 2026 г. по 21 марта 2026 г.

Должность на практике: студент

1. СОДЕРЖАНИЕ ПРАКТИКИ:

1.1. Изучить: процесс записи пациентов в клинику, проблемы при реализации электронной записи в клинику, потенциальны ущерб клиники при отсутствии самозаписи пациентов.

1.2. Практически выполнить: провести обзор и сравнительный анализ существующих систем самостоятельной записи в частные клиники, выявить достоинства и недостатки; построить модель бизнес-процессов записи в поликлинику; определить границы проекта; определить структуру входной и выходной информации, определить источники информации; разработать техническое задание на разработку программного решения; выявить потребности и возможности пользователей при использовании программного решения; выбрать и обосновать архитектуру программного решения; разработать диаграмму в нотации BPMN для процесса посещения врача в частной клинике (от записи, возможной отмены, до посещения врача); разработать диаграмму последовательности в нотации UML для процесса отмены записи в частную клинику; разработать макеты пользовательских интерфейсов и отразить взаимосвязи между ними; разработать план тестирования программного решения.

1.3. Ознакомиться: с актуальными нормативно-правовыми документами и стандартами

2. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ: нет

3. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКИЕ УКАЗАНИЯ: в процессе практики рекомендуется использовать периодические издания и отраслевую литературу годом издания не старше 5 лет от даты начала прохождения практики

Руководитель практики от кафедры
«09» февраля 2026 г. _____ (Бирюкова А.А.)
Подпись

Задание получил
«09» февраля 2026 г. _____ (Щербак М.А.)
Подпись

СОГЛАСОВАНО:
И.О. Заведующего кафедрой:

«09» февраля 2026 г. _____ (Головин С.А.)
Подпись

Проведенные инструктажи:

Охрана труда: «09» февраля 2026 г.

Инструктирующий _____ Головин С.А., и.о. зав. каф. МОСИТ
Подпись

Инструктируемый _____ Щербак М.А.
Подпись

Техника безопасности: «09» февраля 2026 г.

Инструктирующий _____ Головин С.А., и.о. зав. каф. МОСИТ
Подпись

Инструктируемый _____ Щербак М.А.
Подпись

Пожарная безопасность: «09» февраля 2026 г.

Инструктирующий _____ Головин С.А., и.о. зав. каф. МОСИТ
Подпись

Инструктируемый _____ Щербак М.А.
Подпись

С правилами внутреннего распорядка ознакомлен: «09» февраля 2026 г.

_____ Щербак М.А.
Подпись

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»

РТУ МИРЭА

РАБОЧИЙ ГРАФИК
ПРОВЕДЕНИЯПРОИЗВОДСТВЕННОЙ ПРАКТИКИ

студента Щербака М.А. 4 курса группы ИКБО-12-22 очной формы обучения, обучающегося по направлению подготовки 09.03.04 Программная инженерия, профиль «Системная и программная инженерия».

Неделя	Сроки выполнения	Этап	Отметка о выполнении
1	09.02.2026	Подготовительный этап, включающий в себя организационное собрание (Вводная лекция о порядке организации и прохождения производственной практики, инструктаж по технике безопасности, получение задания на практику)	
1-3	10.02.2026-01.03.2026	Исследовательский этап (Обзор предметной области, выделение целевой группы пользователей, формулировка проблем предметной области, обзор и сравнительный анализ существующих систем самостоятельной записи в клинику, выделение достоинств и недостатков; описание предмета исследования; построение модели бизнес-процессов (БП) самозаписи в клинику (нотация IDEF0 состояние «as is»), описание проблем выполнения БП; определение границ проекта, структуры входной и выходной информации, источников информации; постановка задачи на проектирование и разработку; описание взаимодействия с другими системами; разработка технического задания)	
4	02.03.2026	Представление руководителю структурированного материала об объекте и предмете исследования согласно содержанию выше указанного этапа	

4-5	03.03.2026-15.03.2026	Аналитический этап (Построение модели БП записи в клинику (нотация IDEF0, состояние «to be»); определение возможностей пользователей при работе с приложением; определение и обоснование архитектуры программного решения, платформы на которой будет работать программное решение; разработка логической модели базы данных; формирование таблиц сущностей с атрибутами и типами данных; разработка диаграммы в нотации BPMN для процесса посещения врача в частной клинике (от записи, возможной отмены, до посещения врача), диаграммы последовательности в нотации UML для процесса отмены записи в частную клинику, макетов пользовательских интерфейсов и взаимосвязи между ними, план тестирования программного решения для записи в клинику)	
6	16.03.2026	Представление руководителю материала согласно содержанию выше указанного этапа, реализуемого на 4-5 неделях практики; а также иного ранее непредставленного материала предыдущих этапов практики в случае наличия отметок о невыполнении	
6	17.03.2026-20.03.2026	Подготовка окончательной версии отчета по практике (Оформление материалов отчета в полном соответствии с требованиями на оформление письменных учебных работ студентов)	
6	21.03.2026	Представление окончательной версии отчета по практике, загруженного в СДО, руководителю	

Руководитель практики от кафедры _____/ Бирюкова А.А, к.т.н., доцент /

Обучающийся _____/ Щербак М.А./

Согласовано:

И.О. Заведующего кафедрой _____/Головин С.А., д.т.н., профессор/

АННОТАЦИЯ

Отчёт по производственной (учебной) практике посвящён участию в разработке клиент–серверного приложения для самозаписи пациентов частной клиники. В ходе практики были изучены особенности организации процесса записи на приём, проведён анализ существующих программных решений и сформированы требования к созданию собственного программного продукта. Цель практики заключалась в закреплении и развитии профессиональных компетенций в области проектирования и реализации мобильных приложений.

В отчёте представлены результаты анализа предметной области, описаны архитектура клиент–серверного приложения, пользовательский интерфейс, структура базы данных и основные алгоритмы функционирования системы. Отдельное внимание уделено выбору технологий разработки: мобильный клиент реализуется с использованием фреймворка Flutter (язык Dart), серверная часть — на языке Go. Отчёт по практике выполнен на 66 листах, содержит 19 рисунков, 3 таблицы и 1 приложение.

СОДЕРЖАНИЕ

Список используемых сокращений 8

Введение 10

1 Исследовательский раздел 12

1.1. Характеристика предметной области исследования 12

1.2. Анализ существующих аналогов 14

1.3. Техническое задание на разработку 20

1.4 Постановка задачи на разработку 25

2 Проектный раздел 27

2.1. Функции и роли пользователей системы 27

2.1.1. Дерево функций 28

2.1.2. Диаграмма вариантов использования 32

2.2. Информационное обеспечение 34

2.2.1. Логическая модель данных 34

2.2.2. Диаграмма классов анализа 35

2.3. Прототипы пользовательских интерфейсов 36

2.4. Математическое обеспечение 40

2.5. Архитектура программного продукта 46

Список используемых источников 50

Приложения 53

Приложение А. Техническое задание на разработку 54

СПИСОК ИСПОЛЬЗУЕМЫХ СОКРАЩЕНИЙ

API - Application Programming Interface (программный интерфейс приложения)

БД - база данных

BLoC - Business Logic Component (компонент бизнес-логики, паттерн управления состоянием)

ВКР - выпускная квалификационная работа

ГОСТ - государственный стандарт

ДМС - добровольное медицинское страхование

ЕГИСЗ - Единая государственная информационная система в сфере здравоохранения

ИС - информационная система

JSON - JavaScript Object Notation (текстовый формат обмена данными)

JWT - JSON Web Token (токен для аутентификации)

МИС - медицинская информационная система

ОМС - обязательное медицинское страхование

ОС - операционная система

ПМИ - программа и методика испытаний

ПО - программное обеспечение

ПП - программный продукт

REST- Representational State Transfer (архитектурный стиль взаимодействия компонентов)

СУБД - система управления базами данных

TLS - Transport Layer Security (протокол защиты транспортного уровня)

TCO - Total Cost of Ownership (совокупная стоимость владения)

UI - User Interface (пользовательский интерфейс)

UML - Unified Modeling Language (унифицированный язык моделирования)

UX - User Experience (пользовательский опыт)

XSS - Cross-Site Scripting (межсайтовый скриптинг)

FCM - Firebase Cloud Messaging (облачный сервис уведомлений)

HTTP- HyperText Transfer Protocol (протокол передачи гипертекста)

HTTPS - HTTP Secure (защищённый протокол передачи гипертекста)

IDEF0 - Integration Definition for Function Modeling (нотация функционального моделирования)

SMTP - Simple Mail Transfer Protocol (простой протокол передачи почты)

SMS - Short Message Service (служба коротких сообщений)

SQL - Structured Query Language (язык структурированных запросов)

CPU - Central Processing Unit (центральный процессор)

SSD - Solid-State Drive (твердотельный накопитель)

ВВЕДЕНИЕ

Развитие цифровых технологий оказывает значительное влияние на сферу здравоохранения. Для частных медицинских организаций, работающих в условиях высокой конкуренции, особую роль играет качество сервиса и удобство взаимодействия пациента с клиникой. Один из ключевых процессов в этом взаимодействии — запись на приём к врачу. От того, насколько быстро и прозрачно пациент может выбрать врача и время приёма, зависит удовлетворённость сервисом и лояльность к клинике [14]

На практике пациенты часто сталкиваются с рядом проблем: загруженные телефонные линии, ограниченное время работы регистратуры, очереди, ошибки при ручном оформлении записи, недостаток актуальной информации о расписании врачей [2]. Для частных клиник подобные недостатки приводят не только к снижению качества обслуживания, но и к прямым потерям — пациенты уходят к конкурентам, предлагающим более удобные цифровые сервисы.

Актуальность выбранной темы заключается в необходимости автоматизации процесса записи к врачу в частной клинике с использованием современных мобильных и веб-технологий. Разработка клиент–серверного приложения для самозаписи пациентов позволит повысить доступность медицинских услуг, сократить нагрузку на администраторов, минимизировать количество ошибок и улучшить общий уровень сервиса.

Тема разработки клиент–серверного приложения для самозаписи пациентов частной клиники предполагает реализацию распределённой системы, включающей мобильное приложение на базе Flutter (язык Dart) для пациентов и серверное приложение на языке Go, обеспечивающее обработку запросов, управление расписанием врачей и доступ к данным. Целью работы является проектирование и разработка прототипа клиент–серверного приложения, обеспечивающего удобную самозапись пациентов частной клиники на приём к врачу и администрирование расписания со стороны персонала клиники.

Задачи, которые необходимо решить, включают в себя:

- исследование предметной области и выделение актуальных проблем существующей организации записи;
- анализ существующих программных решений для записи к врачу и выявление их преимуществ и недостатков;
- моделирование бизнес-процессов клиники в контексте записи пациентов на приём;
- формирование требований и разработку технического задания на программный продукт;
- проектирование архитектуры клиент–серверного приложения и структуры базы данных;
- выбор инструментов разработки и обоснование технологического стека (Flutter/Dart и Go);
- реализацию прототипа приложения и проведение его тестирования;
- оценку полученных результатов и формулирование выводов.

Объектом исследования является процесс взаимодействия пациента и частной клиники при записи на приём к врачу.

Предметом исследования являются методы и программные средства автоматизации процесса записи пациентов к врачам в формате клиент-серверного приложения

Структура работы определяется поставленной целью и задачами. В исследовательском разделе рассматриваются предметная область и существующие решения, формируется постановка задачи и техническое задание. В последующих разделах представлены проектирование, реализация и тестирование программного продукта.

1 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ

1.1. Характеристика предметной области исследования

Организация процесса записи пациентов к врачу является важной составляющей управления деятельностью медицинского учреждения. Для частной клиники, ориентированной на комфорт пациента и эффективность загрузки специалистов, данный процесс напрямую влияет на качество сервиса и финансовые показатели.

Традиционный способ записи через регистратуру предполагает обращение пациента по телефону или личное посещение клиники. Такой подход сопровождается рядом типичных проблем:

- перегруженность телефонных линий в часы пик [5];
- ограниченный график работы регистратуры;
- высокая вероятность ошибок при ручном внесении данных в журналы или информационную систему [3];
- отсутствие для пациента прозрачной информации о доступных временных слотах приёма [1];
- затруднённое изменение или отмена записи без повторного обращения в регистратуру [4];

Дополнительной особенностью частных клиник является необходимость гибко управлять расписанием специалистов, учитывать различные тарифы, скидочные программы, особенности приёма первичных и повторных пациентов. При ручной обработке записей возрастает риск некорректного распределения нагрузки между врачами, наложения приёмов, потери записей или формирования «окон» в расписании.

Современные тенденции в здравоохранении связаны с цифровизацией услуг: пациенты ожидают возможности самостоятельно выбирать врача и время приёма через личный кабинет, мобильное приложение или веб-сайт. Для частной клиники наличие удобного цифрового сервиса записи становится конкурентным

преимуществом, повышающим лояльность пациентов и снижающим нагрузку на персонал.

Мобильное приложение для самозаписи пациентов позволяет:

- предоставлять пациенту доступ к актуальному расписанию врачей в режиме 24/7;
- уменьшить количество обращений в регистратуру по телефону и лично;
- сократить время оформления записи и сопутствующих уточнений;
- снизить влияние человеческого фактора при работе с расписанием;
- обеспечить пациенту удобные механизмы изменения и отмены записи;
- собирать данные для последующего анализа загрузки врачей и планирования.

Таким образом, предметной областью работы является автоматизация процесса записи пациентов к врачам частной клиники с использованием клиент–серверного программного обеспечения и мобильного клиента. Выбор данного технологического подхода обусловлен принципиальными различиями в моделях функционирования государственных и частных медицинских учреждений, а также современными потребительскими трендами.

Различия процессов записи в государственных и частных клиниках

В государственных учреждениях запись строится по модели «занятость ресурса»: пациент подстраивается под доступные слоты в рамках ОМС, его роль пассивна, а успех измеряется фактом получения талона. В частной клинике действует модель «клиентский сервис»: пациент — активный потребитель, выбирающий удобное время, врача и соотношение цены и комфорта. Идентификация здесь привязана к номеру телефона как ключу в CRM-системе, а успех записи определяется не просто её фактом, а удобством для пациента, формирующим лояльность.

Эти различия напрямую влияют на требования к автоматизации. В государственной системе автоматизация направлена на распределение потока

пациентов. В частной медицине она должна решать задачи удержания клиента: сокращать время на запись и повышать качество сервиса, так как в условиях конкуренции именно удобство взаимодействия становится решающим фактором выбора клиники.

Обоснование необходимости мобильного клиента

Выбор мобильного приложения в качестве клиентской части системы продиктован следующими факторами:

- Push-уведомления для снижения неявок. Мобильный клиент позволяет отправлять автоматические напоминания о визите за 24 часа и за 2 часа. Это напрямую решает проблему неявок, которая, по данным официальных источников, достигает 30–36%, освобождая слоты для других пациентов.
- Скорость и удобство идентификации. Авторизация по номеру телефона избавляет пациента от необходимости каждый раз вводить данные или диктовать их сотруднику регистратуры, что снижает риск ошибок ручного ввода, характерных для телефонной записи.
- Соответствие ожиданиям аудитории. Приложение всегда под рукой, что критично для оперативной записи или переноса визита. Для платежеспособной аудитории частных клиник отсутствие мобильного сервиса сегодня воспринимается как признак технологической отсталости и снижает лояльность.

Таким образом, клиент-серверная архитектура с мобильным клиентом обеспечивает пациенту максимальное удобство, а клинике — снижение нагрузки на регистратуру и сокращение числа неявок за счет автоматических напоминаний.

1.2. Анализ существующих аналогов

На рынке представлены различные программные решения, позволяющие организовать запись пациентов к врачу. Рассмотрим три популярных класса решений и типичных представителей этих классов.

1. Госуслуги «Запись к врачу».

Это государственный сервис, интегрированный с региональными медицинскими информационными системами. Функциональные возможности: выбор медицинской организации, врача и времени приёма, просмотр истории посещений.

Преимущества:

- интеграция с государственной системой здравоохранения;
- доступность для большинства государственных поликлиник;
- единая точка входа для ряда государственных услуг.

Недостатки:

- перегруженный и не всегда интуитивный интерфейс;
- ограниченная функциональность с точки зрения частных клиник;
- зависимость от региональной инфраструктуры и технические сбои.

2. Коммерческие сервисы записи (например, «СберЗдоровье»).

Данные платформы предоставляют единый каталог клиник и врачей с возможностью записи онлайн. Функциональные возможности: поиск врачей по специализации и геолокации, онлайн-запись, отзывы пациентов, иногда — консультации в чате и телемедицина.

Преимущества:

- современный интерфейс;
- широкий выбор клиник и специалистов;
- дополнительные сервисы (отзывы, рейтинги, онлайн-консультации).

Недостатки:

- часть функциональности доступна только при оплате или регистрации в экосистеме поставщика;
- требования по интеграции и условиям сотрудничества, которые могут быть невыгодны небольшой частной клинике;
- акцент на marketplace-модель, а не на внутренние процессы конкретной клиники.

3. Рынок программного обеспечения для медицинских учреждений представлен двумя основными категориями решений:
    - Комплексные медицинские информационные системы (МИС). Это корпоративные продукты, автоматизирующие полный цикл работы клиники: от записи и электронных карт до складского учета и отчетности. К наиболее функциональным МИС, подходящим для частных клиник разного масштаба, относятся: MEDMIS, Medesk, ArchiMed+, КМИС («Комплексная медицинская информационная система»), «МЕДИАЛОГ».
    - Сервисы онлайн-записи и CRM. Данный класс решений (например, Yclients) ориентирован прежде всего на малый бизнес и сферу услуг. Они обеспечивают простой и быстрый старт, удобный интерфейс для записи, но, как правило, уступают МИС в возможностях ведения строгой медицинской документации, интеграции с лабораториями и государственными системами (ЕГИСЗ). Выбор конкретного типа системы зависит от масштаба клиники. Если крупные сети («Медси») могут позволить себе разработку уникального ПО «с нуля» под свои процессы, то для небольших и средних частных клиник оптимальным решением является внедрение готовой МИС, предлагающей сбалансированный функционал.

Это платформы для автоматизации бизнеса, в том числе медицинских клиник.

Функциональные возможности: управление расписанием специалистов, онлайн-запись, напоминания клиентам, отчётность и аналитика.

Преимущества:

- гибкая настройка расписания и прав доступа;
- наличие инструментов аналитики и маркетинга;
- поддержка уведомлений и интеграций.

Недостатки:

- высокая стоимость для небольших клиник;
- избыточный функционал, не всегда востребованный в конкретной организации;
- необходимость обучения персонала и администрирования системы.

Для объективного сравнения существующих решений необходимо выделить критерии, которые напрямую влияют на достижение цели работы — создание эффективной системы записи, снижающей нагрузку на персонал и повышающей удовлетворенность пациентов.

1. Наличие механизма самостоятельной записи пациентов (онлайн-запись) — это ключевое требование к системе. Механизм самозаписи позволяет пациенту выбрать удобное время без участия регистратуры, что напрямую решает проблему перегруженности телефонных линий и очередей.

Показатель оценки: поддерживает ли система запись через разные каналы: веб-версия (сайт клиники), мобильное приложение, виджеты на сторонних площадках. Важно, чтобы выбранное пациентом время сразу блокировалось в расписании, исключая двойные бронирования.

2. Соответствие интерфейса принципам проектирования пользовательского опыта - субъективное «удобство» невозможно измерить. Но можно оценить, насколько интерфейс соответствует устоявшимся стандартам юзабилити, что косвенно гарантирует его понятность для массового пользователя.

Показатель оценки: Соответствие интерфейса распространенным паттернам (например, стандартным элементам мобильных ОС), наличие/отсутствие скриншотов интерфейса в открытом доступе, возможность протестировать демоверсию.

3. Возможность настройки бизнес-процессов клиники без программирования - частные клиники имеют уникальные процессы: у одних требуется предоплата за запись, у других — подтверждение записи администратором после онлайн-заявки, третьи работают только по предварительному согласованию с врачом.

Показатель оценки: Наличие гибких настроек в системе: можно ли задать разные типы приема (первичный/повторный), привязать услуги к конкретным врачам, настроить напоминания, правила отмены записи и лимиты предварительной записи.

4. Совокупная стоимость владения (TCO) за три года - стоимость сама по себе ни о чем не говорит без привязки к масштабу бизнеса. Для небольших частных клиник (до 10 врачей) ежемесячная выручка составляет в среднем 1,5–3 млн рублей. Инвестиции в ИТ на уровне 5–10% от годовой прибыли считаются нормальными, выше — рискованными.

Показатель оценки: Сумма затрат за три года включает: стоимость лицензий (разовый платеж или ежемесячная подписка), затраты на внедрение и настройку, стоимость технической поддержки, затраты на интеграции и возможное дооборудование (серверы, терминалы). Низкой стоимостью можно считать до 150–200 тыс. руб. за три года для небольшой клиники, средней — до 500 тыс., высокой — свыше 500 тыс. или процент с оборота (если система берет комиссию с каждой записи).

5. Наличие и сложность интеграции с внешними сервисами - система записи не существует в вакууме. Для полноценной работы частной клиники требуются интеграции с:

- Телефонией (автоматическое создание записи при звонке, запись разговоров);
- Платежными системами (онлайн-оплата при записи);
- Лабораторными информационным системами (передача результатов анализов в карту пациента);
- Системами рассылок (email, SMS, MAX);
- ЕГИСЗ (если клиника планирует работать с ОМС/ДМС).

Показатель оценки: Наличие готовых модулей (коннекторов) к популярным сервисам (например, АТОЛ, Сбербанк, Инвитро, 1С) против необходимости разработки интеграций «под ключ» силами программистов. Сложность интеграции

оценивается как «низкая» (есть готовая интеграция), «средняя» (требуется настройка API специалистом) или «высокая» (требуется разработка индивидуального решения).

6. Наличие мобильного приложения для пациентов - как обосновывалось ранее, мобильное приложение обеспечивает максимальное удобство для пациента (push-уведомления, быстрый доступ, геолокация) и решает проблему неявок. Отсутствие приложения снижает конкурентоспособность клиники.

Показатель оценки: есть ли у системы собственное мобильное приложение (или возможность создать «белое» приложение под брендом клиники). Оценивается функционал приложения: только запись или также просмотр истории, оплата, телемедицина. Важный нюанс: некоторые МИС предлагают мобильное приложение как отдельную платную опцию.

Был проведен сравнительный анализ типовых решений, озвученных ранее. Результаты оценки по установленным критериям представлены в таблице 1.1.

Таблица 1.1 — Сравнительная характеристика существующих решений для записи к врачу

Название программного продукта	Механизм самозаписи пациентов	Адаптация под нужды конкретной частной клиники	Стоимость внедрения и эксплуатации	Необходимость сложной интеграции с внешними сервисами	наличие мобильного приложения, ориентированного на пациентов
Госуслуги «Запись к врачу»	+	-	+	-	+
«СберЗдоровье»	+	-	+	+	+
Корпоративные системы управления расписанием (например, Yclients и аналоги)	+	+	+	+	+
Разрабатываемое клиент–серверное приложение	+	+	-	-	+

Сводный сравнительный анализ показывает, что:

- государственные сервисы ориентированы преимущественно на государственные медицинские учреждения и не учитывают особенности частных клиник;
- крупные коммерческие платформы предоставляют широкий функционал, но требуют дополнительных затрат и не всегда позволяют реализовать индивидуальные сценарии работы клиники - например, автоматизированную интеграцию с лабораторным оборудованием или кастомные протоколы приёма пациентов по графику специалиста;
- Корпоративные системы управления расписанием нередко обладают избыточной функциональностью и объединяют множество бизнес-процессов (HR, финансы, логистика), выходящих за рамки задачи данной работы, что усложняет внедрение, повышает затраты и снижает удобство для медицинского персонала.

Ни одно из рассмотренных решений не обеспечивает одновременно:

- полного учёта внутренних процессов конкретной частной клиники;
- простоты и доступности для пациентов (ЕМИАС удобен для госклиник по ОМС, но не адаптирован для частных);
- возможности гибкой доработки функционала под нужды конкретного заказчика без привязки к крупному внешнему сервису.

Это обосновывает необходимость разработки собственного клиент–серверного приложения для самозаписи пациентов частной клиники.

1.3. Техническое задание на разработку

Выявленные в ходе анализа предметной области и существующих аналогов пробелы в функциональности определяют чёткий набор требований к разрабатываемому программному продукту. Для последующей реализации системы самозаписи пациентов необходимо формализовать эти требования в виде технического задания на разработку клиент–серверного приложения.

Ключевым ориентиром при формировании технического задания является целевая модель бизнес-процессов, описываемая диаграммой «to be». Эта модель показывает, каким образом изменится организация записи пациентов к врачу после внедрения системы. На рисунке 1.1 представлена контекстная диаграмма бизнес-процесса «Работа системы самозаписи пациентов частной клиники» в нотации IDEF0 в вариации «to be».

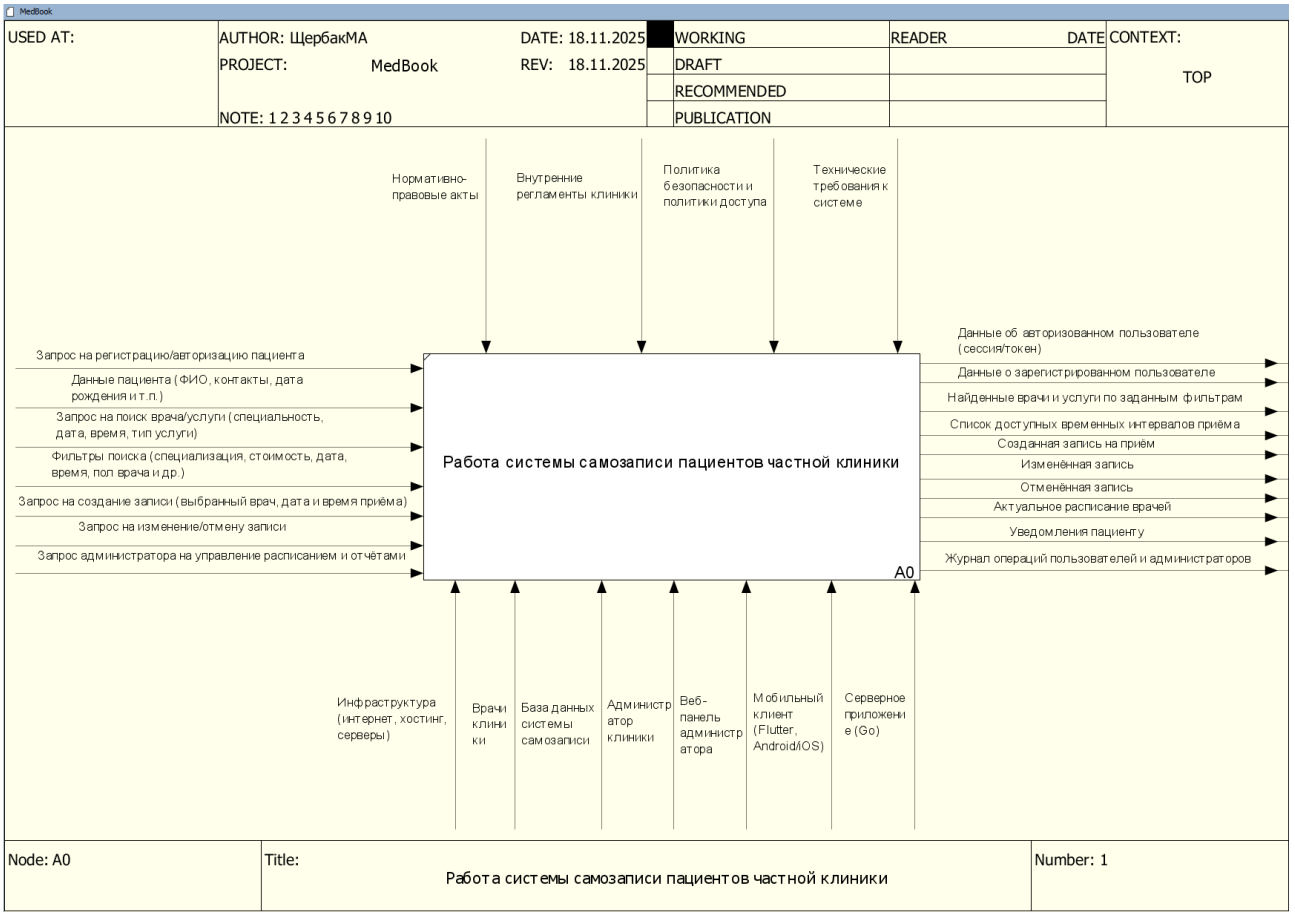

Рисунок 1.1 — Контекстная диаграмма бизнес-процесса «Работа системы самозаписи пациентов частной клиники»

Взаимодействие пользователя с системой начинается с регистрации и авторизации. После успешной идентификации пациент получает доступ к личному кабинету, в котором он может просматривать и редактировать свои персональные данные. Далее пользователь формирует запрос на поиск врача или услуги, задаёт фильтры (специализация, дата, время, стоимость приёма) и получает список доступных специалистов и временных интервалов. На основе полученных данных

пациент оформляет запись на приём, а при необходимости может изменить время визита или отменить запись с учётом регламентов клиники.

Администратор клиники посредством веб-панели управляет расписанием врачей, открывает и закрывает временные слоты, контролирует корректность записей, формирует отчёты по загрузке специалистов и обращаемости пациентов. Все значимые операции пользователей и администратора фиксируются в журнале, что обеспечивает прослеживаемость действий и повышает информационную безопасность системы.

Ключевым отличием целевой модели «to be» от текущего состояния предметной области «as is» является комплексная трансформация, затрагивающая несколько основных аспектов:

- Круглосуточная кроссплатформенная доступность записи.

  Вместо обращения в регистратуру по телефону или при личном визите пациенты получают возможность самостоятельно записываться к врачу через мобильное приложение, доступное в любое время. Использование кроссплатформенного фреймворка Flutter обеспечивает единый пользовательский опыт на основных мобильных платформах и позволяет в перспективе реализовать веб-версию клиента.
- Интеграция с внутренним расписанием клиники и снижение влияния человеческого фактора.

  Все операции по созданию, изменению и отмене записей выполняются в единой базе данных и автоматически отражаются в расписании врачей. Это уменьшает риск накладок и потери записей, снижает нагрузку на персонал регистратуры и обеспечивает руководству клиники актуальные аналитические данные для управления загрузкой специалистов.

Сформированная на основе целевой модели совокупность требований служит основой для разработки технического задания на создание клиент–серверного приложения для самозаписи пациентов частной клиники.

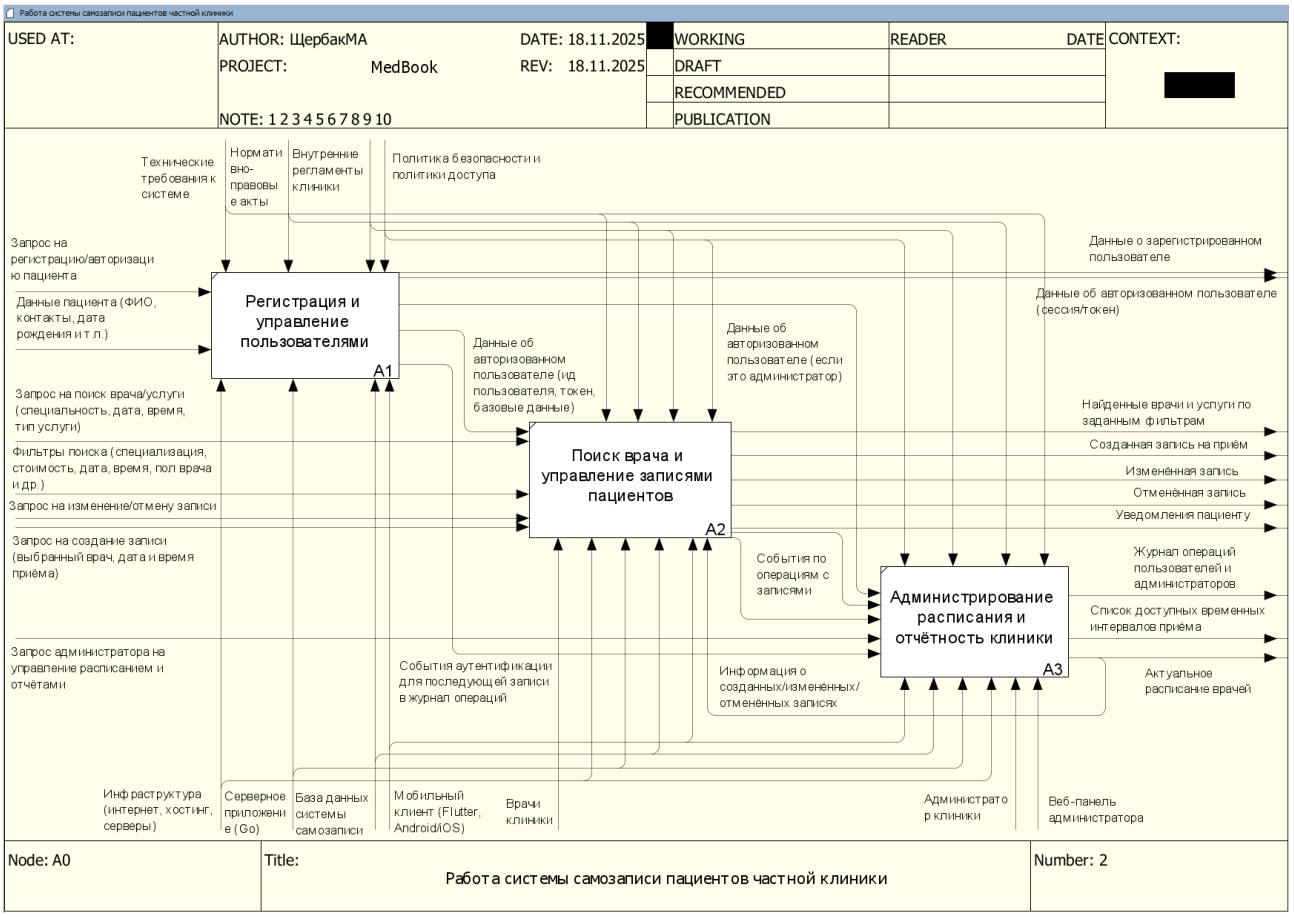

Рисунок 1.2 — Декомпозиция контекстной диаграммы

Начальные данные формируются непосредственно в процессе эксплуатации системы самозаписи пациентов. Основным их источником выступают действия двух групп пользователей:

- пациенты вводят данные регистрации и авторизации, выбирают врачей и услуги, формируют запросы на поиск и создают, изменяют или отменяют записи на приём;
- администраторы клиники задают и корректируют расписание врачей, обрабатывают обращения пациентов, формируют отчётность.

К предустановленным данным относятся справочники, которые загружаются в систему при её первичной настройке: перечень врачей и их специализаций, виды медицинских услуг, рабочие графики по умолчанию, статусы записей, параметры уведомлений и основные настройки клиники.

Основными выходными данными являются:

- для пациента: список доступных врачей и услуг, расписание с доступными временными интервалами, подтверждённые и изменённые записи, уведомления о предстоящих приёмах и изменениях, обновлённый профиль пользователя;
- для администратора: актуальное расписание врачей и кабинетов, список активных и отменённых записей, отчёты о загрузке специалистов и обращаемости пациентов, журнал операций пользователей и персонала.

Взаимодействие с внешней средой построено по клиент–серверной архитектуре. Мобильный клиент и веб-панель администратора обращаются к серверной части по REST API через защищённый протокол HTTPS. Обмен данными осуществляется в формате JSON, а для аутентификации и авторизации используются JWT-токены. Серверное приложение, в свою очередь, взаимодействует с СУБД для хранения структурированных данных о пользователях, расписании и записях на приём.

Реализация проекта сопряжена с рядом рисков, для управления которыми сформирована матрица рисков, представленная в таблице 1.2.

Таблица 1.2 — Матрица рисков

Риск	**Вероятность**	**Последствия**	**Меры снижения риска**
Перегрузка сервера и снижение доступности сервиса	Средняя	Невозможность записи, потеря части пользователей	Оптимизация запросов, масштабируемая архитектура, использование облачной инфраструктуры
Утечка персональных данных пациентов	Низкая	Нарушение законодательства, репутационные потери	Шифрование данных, разграничение доступа, использование защищённых протоколов, регулярные аудиты безопасности
Сбои при работе мобильного приложения	Средняя	Невозможность завершить запись, снижение доверия к приложению	Тестирование на различных устройствах, мониторинг ошибок, выпуск обновлений
Низкая удобство интерфейса для пациентов	Средняя	Снижение популярности приложения, отказ от	Проектирование UX, проведение опросов и юзабилити-тестирования

		использования сервиса	
Несовместимость приложения с частью устройств	Низкая	Часть пациентов не сможет установить приложение	Использование кроссплатформенного фреймворка, тестирование на разных версиях ОС
Ошибки персонала при работе с административным интерфейсом	Низкая	Некорректное расписание, конфликты приёмов	Обучение персонала, разработка понятного интерфейса, валидация данных

В соответствии с данным описанием написано техническое задание по ГОСТ 19.201-78. «Единая система программной документации. Техническое задание. Требование к содержанию и оформлению», которое приведено в Приложении А.

1.4 Постановка задачи на разработку

На основании характеристики предметной области, анализа существующих решений и моделирования бизнес-процессов можно сформулировать задачу на разработку программного продукта.

Функционально разрабатываемое клиент–серверное приложение должно обеспечивать:

- регистрацию и аутентификацию пациентов;
- просмотр перечня врачей и их расписания;
- выбор времени приёма и создание записи;
- просмотр и управление своими записями (перенос, отмена в установленных рамках);
- взаимодействие с серверной частью через защищённый программный интерфейс;
- административные функции для персонала клиники (управление расписанием, просмотр записей пациентов).

К нефункциональным требованиям относятся:

- удобный и интуитивно понятный пользовательский интерфейс;

- кроссплатформенность мобильного клиента (поддержка основных мобильных платформ);
- обеспечение конфиденциальности и целостности медицинских данных;
- устойчивость работы при увеличении числа пользователей;
- возможность дальнейшего масштабирования и развития функционала.

Формулировка задачи: разработать клиент–серверное приложение, обеспечивающее автоматизацию процесса самозаписи пациентов частной клиники к врачу, включающее мобильный клиент на Flutter/Dart и серверную часть на Go, интегрированное с внутренним расписанием клиники и поддерживающее административные функции.

2 ПРОЕКТНЫЙ РАЗДЕЛ

Проектный раздел посвящен логическому проектированию клиент-серверного приложения для самозаписи пациентов частной клиники. Включает в себя описание информационного, математического и технического обеспечения, архитектуру разрабатываемого продукта.

В этом разделе проектируется архитектура и логика работы программного продукта (ПП), определяются требования к информационному, техническому и математическому обеспечению, проектируется логическая модель базы данных, определяются основные роли в системе и их права доступа к данным. На основе результатов исследовательского раздела формируются функциональные спецификации системы, моделируются бизнес-процессы и пользовательские сценарии взаимодействия без привязки к конкретным технологиям реализации.

2.1. Функции и роли пользователей системы

Логика работы системы позволяет понять, какие функции должны быть разработаны и кто будет их выполнять. Для проектирования логики работы программного продукта определен основной функционал, распределенный по ролям пользователей: пациенты, администраторы клиники и врачи. На основе анализа предметной области сформированы дерево функций и диаграмма прецедентов (вариантов использования) UML.

Для детального описания динамики работы добавлена диаграмма последовательности ключевого сценария «Создание записи».

Роли пользователей

- Пациент: регистрация по телефону, поиск врачей, создание/отмена записей, просмотр истории.
- Администратор: управление расписанием, подтверждение записей, отчеты по загрузке.

- Врач: просмотр своих записей, корректировка расписания.

2.1.1. Дерево функций

Дерево функций используется для наглядного представления функциональной структуры разрабатываемого программного решения и отражает иерархию функций системы автоматизации записи пациентов к врачу. Данный метод позволяет структурировать функциональные возможности приложения, определить основные и вспомогательные функции, а также установить связи между ними.

На верхнем уровне дерева функций располагается главная функция системы - автоматизация процесса записи пациентов к врачу. Данная функция реализуется посредством совокупности подфункций, обеспечивающих взаимодействие различных категорий пользователей с системой.

Основные функциональные блоки системы включают:

1. **Управление пользователями (рис. 2.1)**

   - регистрация пользователей;
   - авторизация и аутентификация;
   - управление ролями (пациент, врач, администратор);
   - редактирование профиля пользователя.

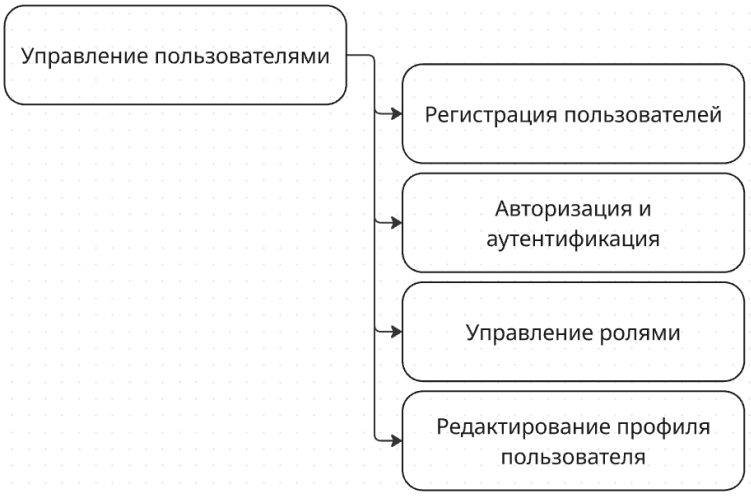

Рисунок 2.1 – Дерево функций управления пользователями

2. **Работа пациента с системой (рис. 2.2)**

- поиск врачей по специализации и другим параметрам;
- просмотр информации о враче;
- просмотр расписания приёма;
- выбор даты и времени приёма;
- создание записи на приём;
- перенос и отмена записи;
- просмотр истории записей;
- получение уведомлений о статусе записи.

Рисунок 2.2 – Дерево функций работы пациента с системой

3. **Функции врача (рис. 2.3)**

- просмотр личного расписания;
- просмотр списка записанных пациентов;
- изменение статуса приёма (проведён, отменён);
- добавление комментариев к приёму.

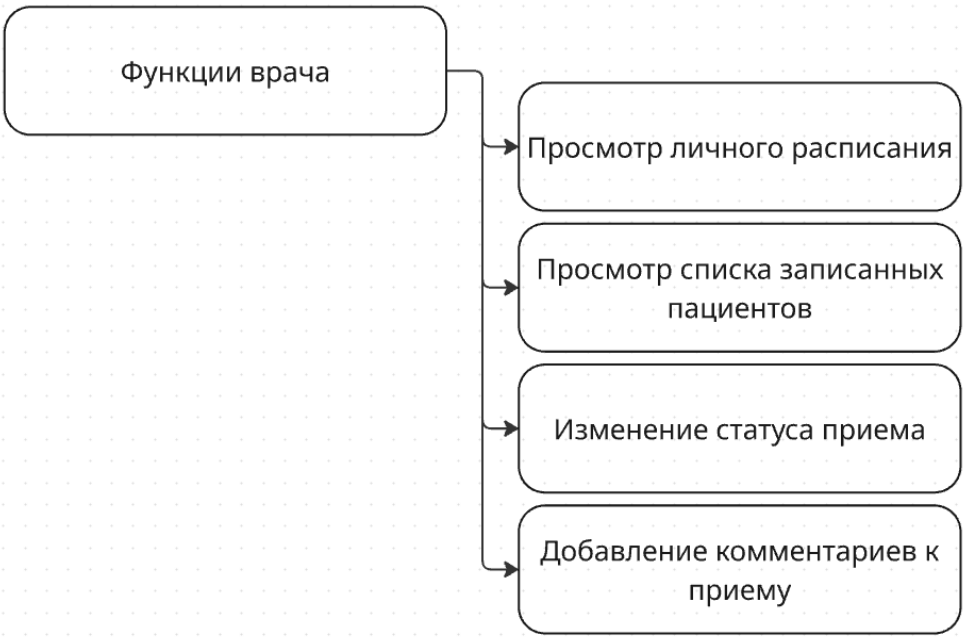

Рисунок 2.3 – Дерево функций врача

4. **Функции администратора (рис. 2.4)**

- управление пользователями системы;
- создание и редактирование расписаний врачей;
- управление медицинскими услугами;
- контроль и корректировка записей пациентов;
- формирование отчётности и статистики.

Рисунок 2.4 – Дерево функций администратора

5. **Системные функции (рис. 2.5)**

- хранение и обработка данных;
- обеспечение безопасности и защиты персональных данных;
- ведение журналов действий пользователей;
- отправка уведомлений (push, e-mail);
- обеспечение надёжности и отказоустойчивости системы.

Рисунок 2.5 – Дерево системных функций

2.1.2. Диаграмма вариантов использования

Диаграмма прецедентов (рисунок 2.6) отражает основные сценарии взаимодействия пользователей с системой записи к врачу. В системе выделяются три категории участников:

- **Пациент** - основной пользователь, который может просматривать расписание врачей, записываться на приём, отменять запись, просматривать историю посещений и получать уведомления о предстоящем визите.

- **Врач** - имеет возможность просматривать собственное расписание, подтверждать приёмы, а также вносить результаты осмотров и медицинские записи в систему.

- **Администратор** - управляет пользователями, создаёт и редактирует расписания врачей, контролирует записи пациентов и обеспечивает корректность функционирования системы.

Таким образом, диаграмма демонстрирует структуру поведения участников системы, разграничивая права и доступ к различным функциям. Пациент ориентирован на взаимодействие с функционалом записи и уведомлений, врач - на подтверждение и ведение медицинских данных, администратор - на организационное и управленческое сопровождение системы.

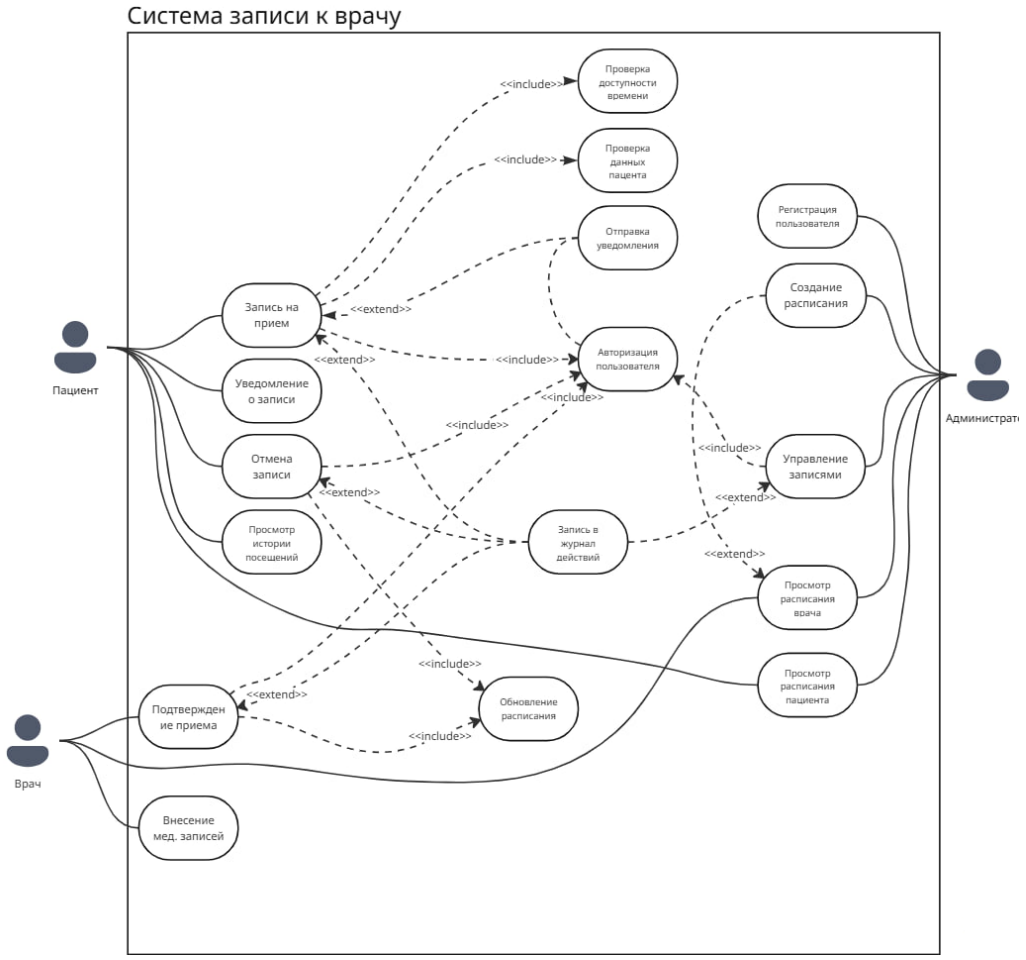

Рисунок 2.6 – диаграмма прецендентов

2.2. Информационное обеспечение

Информационное обеспечение программного продукта представляет собой совокупность методов и средств организации, хранения и управления данными, необходимых для функционирования системы самозаписи пациентов. В контексте клиент-серверного приложения информационное обеспечение определяет, какие данные должны храниться, как они структурированы, кто имеет к ним доступ и какие операции с ними допустимы.

На основе анализа предметной области (раздел 1.1), выявленных требований (раздел 1.3) и спроектированной функциональной структуры (дерево функций, раздел 2.1.1; диаграмма вариантов использования, раздел 2.1.2) были определены основные информационные объекты системы и их взаимосвязи.

2.2.1. Логическая модель данных

Логическая модель базы данных описывает структуру информации, необходимой для функционирования системы, без привязки к конкретной СУБД. Логическая модель служит основой для проектирования физической модели и последующей реализации базы данных.

На рисунке 2.7 представлена логическая модель БД, содержащая эти таблицы.

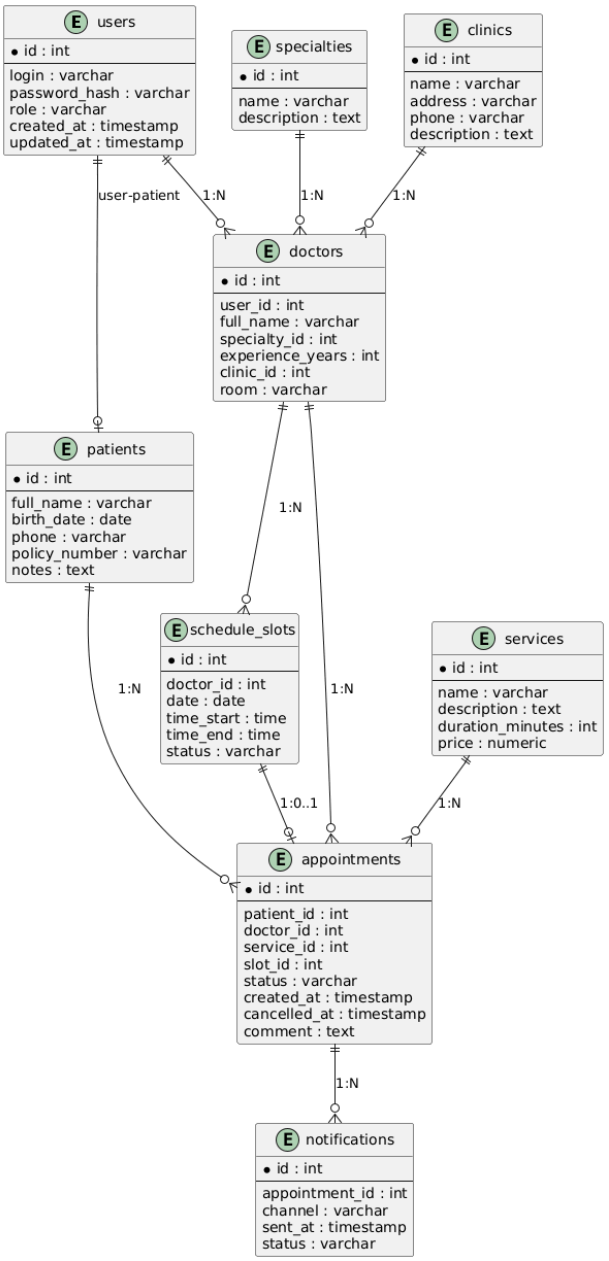

Рисунок 2.7 – Логическая модель БД

2.2.2. Диаграмма классов

На основе логической модели данных и выявленных функциональных требований была разработана диаграмма классов анализа, отражающая статическую структуру системы самозаписи пациентов. Диаграмма (Рис. 2.8) представляет ключевые сущности предметной области, их атрибуты, методы и взаимосвязи, необходимые для реализации функционала, описанного в дереве функций (раздел 2.1.1) и диаграмме вариантов использования (раздел 2.1.2).

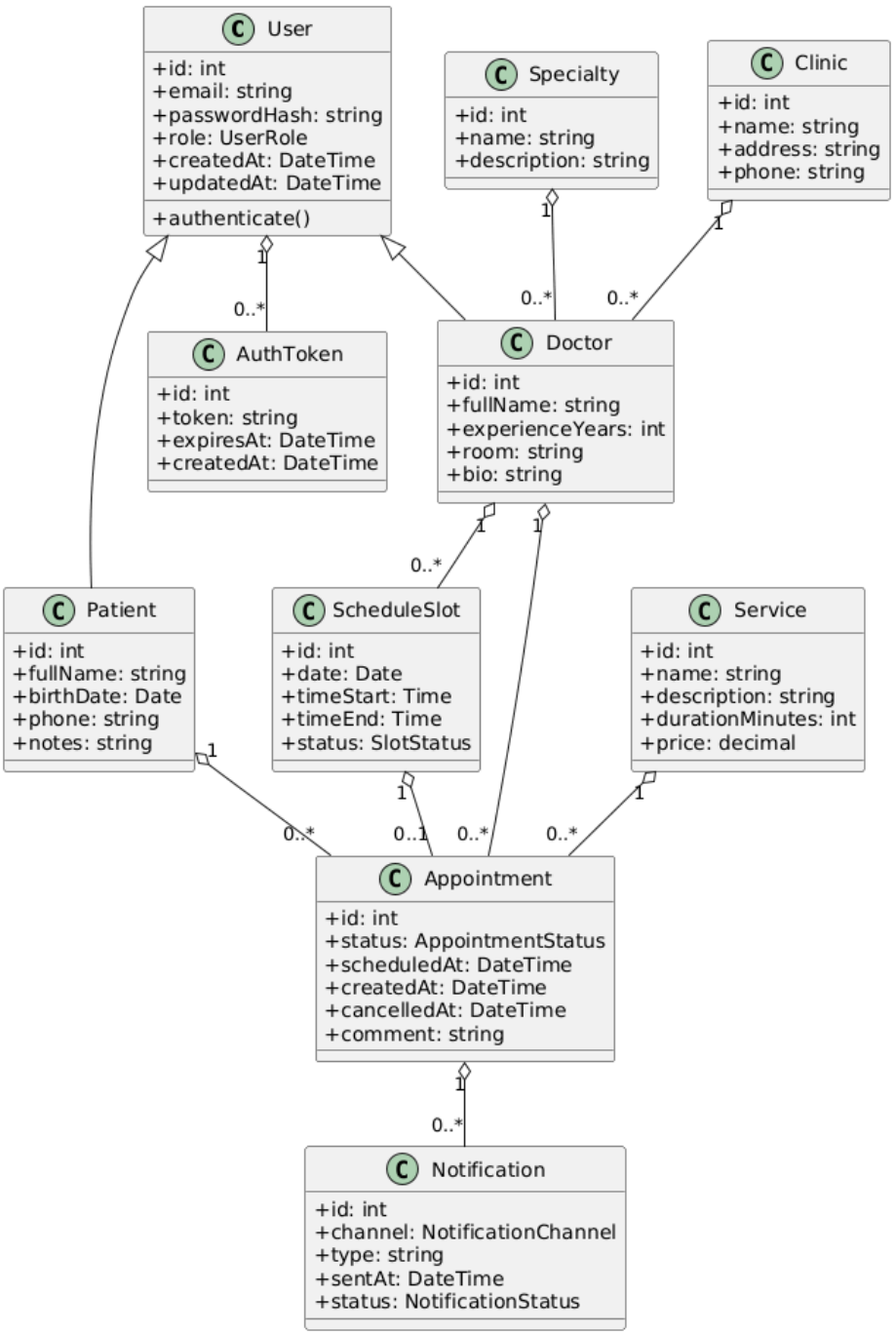

Рисунок 2.8 – диаграмма классов

2.3. Прототипы пользовательских интерфейсов

Перед разработкой клиентской части мобильного приложения был создан макет дизайна в Figma. Дизайн формировался с целью обеспечения удобного и интуитивно понятного интерфейса. На рисунке ниже представлен прототип интерфейса приложения. (Рисунок 2.9)

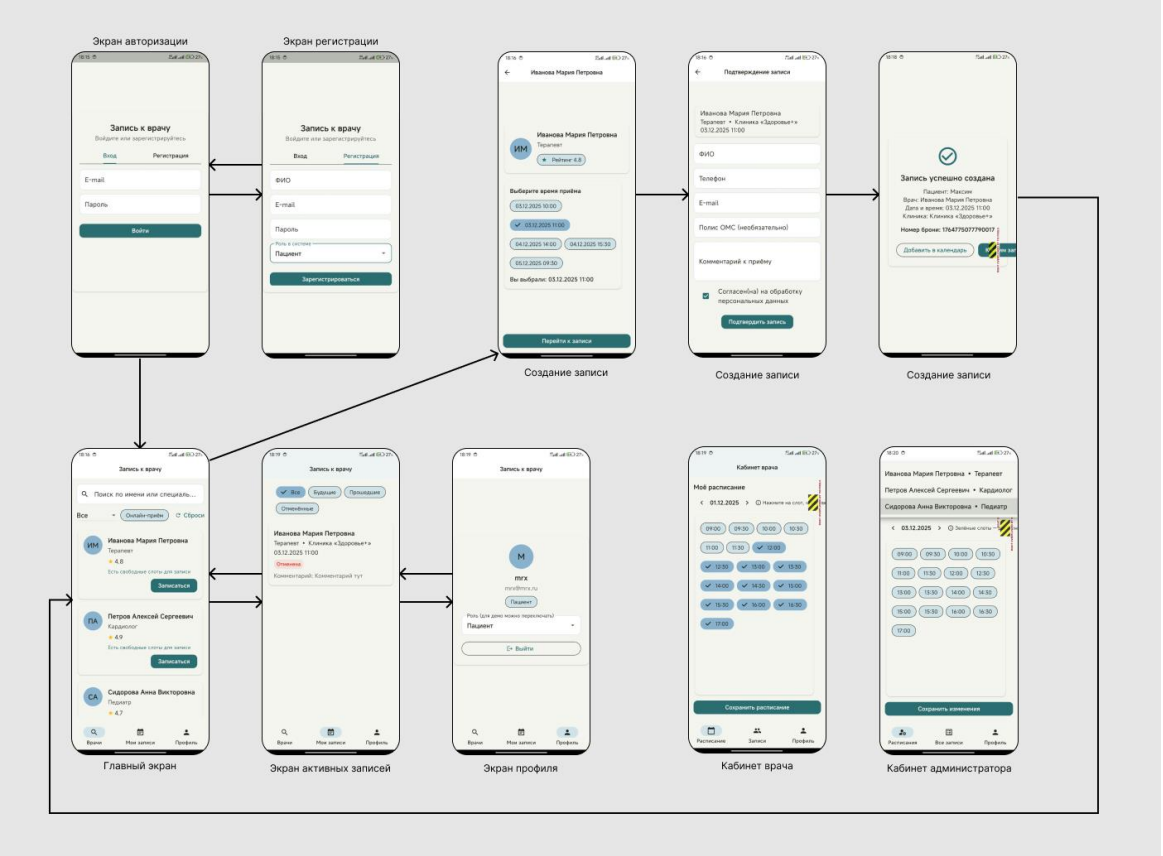

Рисунок 2.9 – Макет пользовательского интерфейса

Рассмотрим межэкранное взаимодействие.

При запуске приложения пользователь сразу попадает на экран авторизации (Рис 2.10). Этот экран служит отправной точкой для работы с системой. Здесь пользователь видит два поля ввода (логин/email и пароль), кнопку "Войти", а также текстовую ссылку "Зарегистрироваться" в нижней части экрана. Если пользователь новый или хочет создать дополнительный аккаунт, он нажимает "Зарегистрироваться".

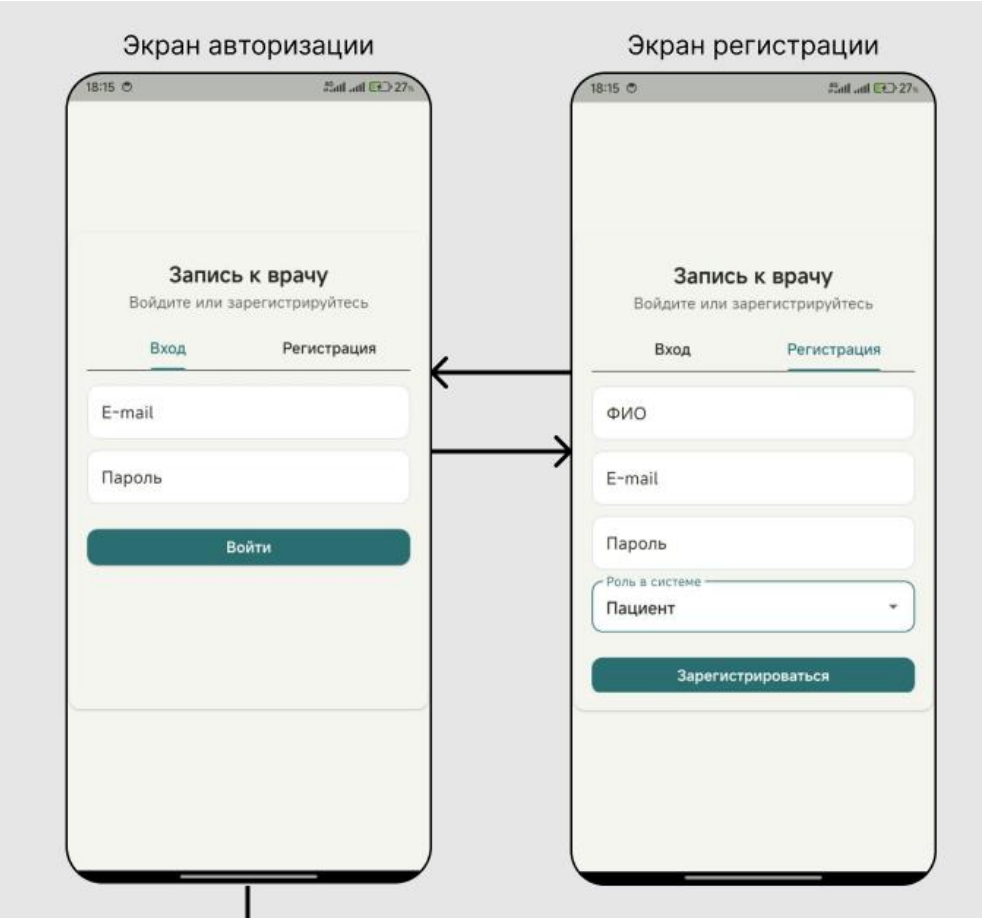

Рисунок 2.10 – Экраны авторизации/регистрации

Это действие плавно переводит его на экран регистрации с анимацией сдвига слева направо. На экране регистрации представлена расширенная форма: поля для имени, email, пароля и подтверждения пароля, кнопка "Создать аккаунт", а также малозаметная ссылка "Уже есть аккаунт?" под основной формой. При успешной авторизации или регистрации пользователь попадает в главное меню.

Пользователь, находясь в Главном меню, может перейти в любой из разделов: Для этого в главном меню расположены соответствующие кнопки. Переходы выполняются с анимацией сдвига экрана влево или вправо.

Главное меню изображено на рисунке 2.11

Рисунок 2.11– Макет главного экрана

На рисунке 2.12 показан экран выбора даты и времени приёма. Здесь используется компонент календаря для выбора дня приёма и список доступных временных слотов. Уже занятые интервалы отображаются как недоступные. После выбора времени пользователь переходит на экран подтверждения записи.

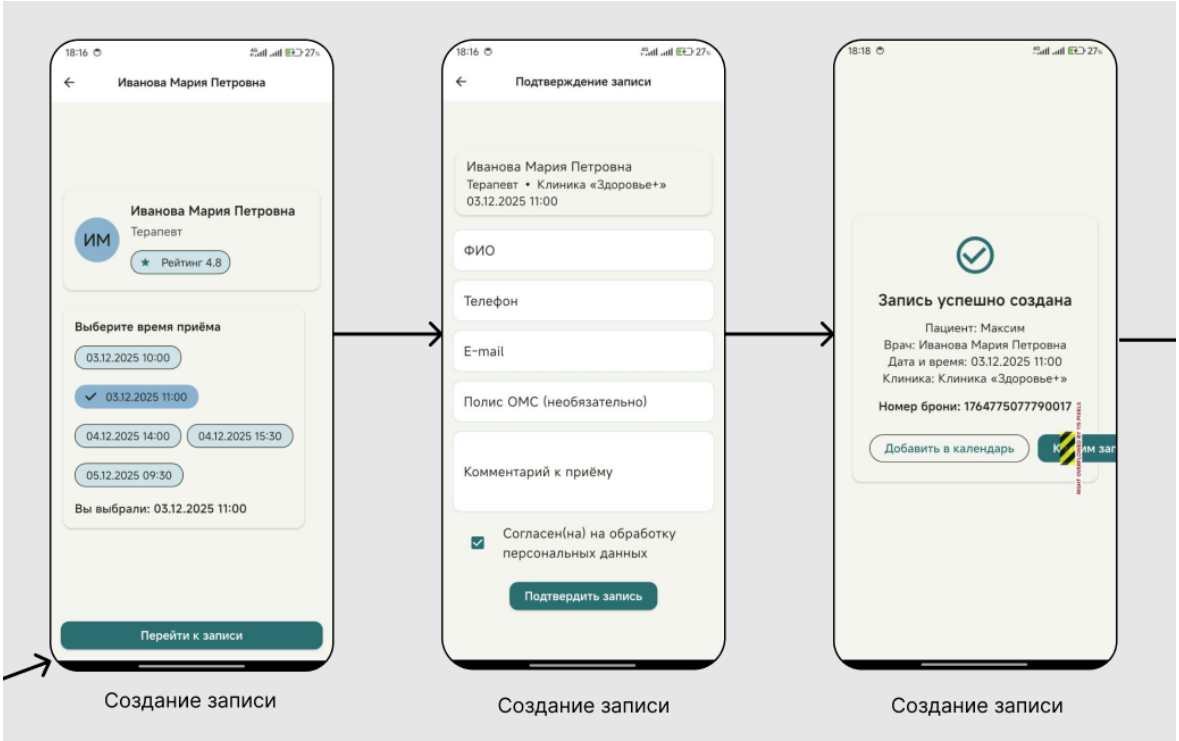

Рисунок 2.12 – Создание записи

2.4. Математическое обеспечение

Математическое обеспечение разрабатываемой системы включает в себя алгоритмы, необходимые для реализации основных функций: поиска свободного времени, проверки доступности слотов, предотвращения конфликтов при записи и отправки уведомлений. В данном разделе представлены блок-схемы ключевых алгоритмов и оценка сложности их выполнения.

Алгоритм проверки доступности временного интервала

При попытке записи на приём система должна убедиться, что выбранное время не занято. Алгоритм реализует проверку пересечения нового интервала с существующими записями.

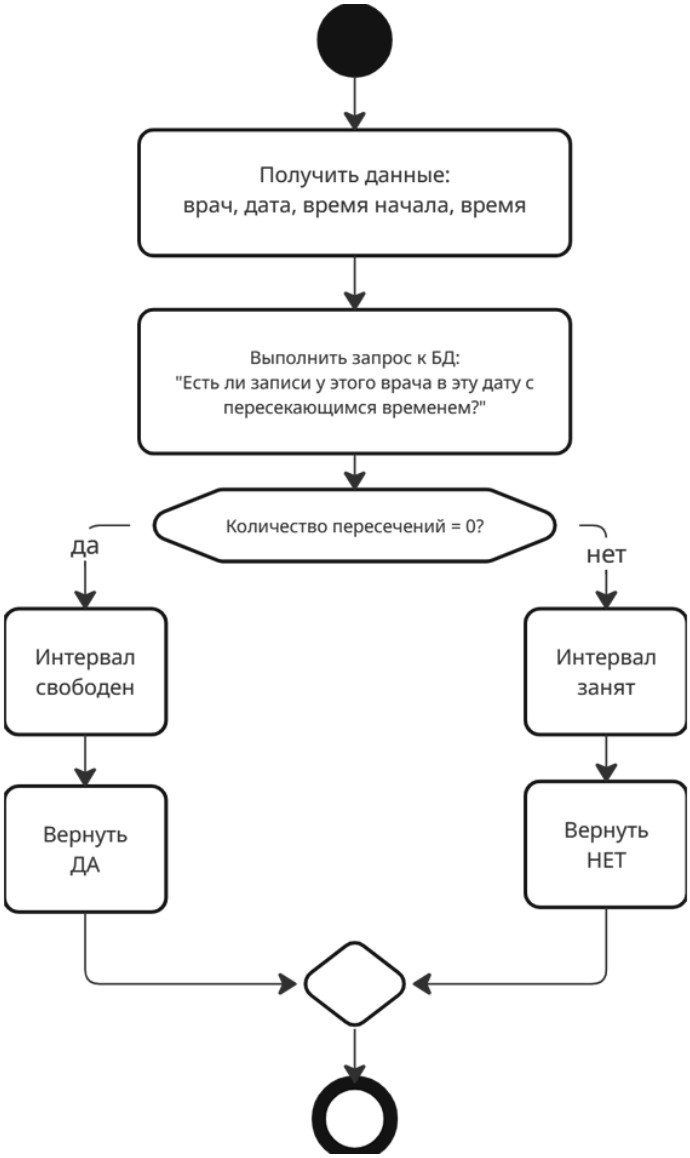

Рисунок 2.13 – Блок-схема алгоритма проверки доступности временного интервала

Оценка сложности: При наличии индексов в базе данных сложность алгоритма составляет O(log n), где n — общее количество записей. Это означает,

что даже при росте числа записей время выполнения увеличивается незначительно.

Алгоритм поиска ближайших свободных слотов

Когда пациент ищет, когда можно записаться к врачу, система должна показать ближайшие доступные окна.

Рисунок 2.14 – Блок-схема алгоритма поиска ближайших свободных слотов

Алгоритм предотвращения двойной записи

Особенность онлайн-систем в том, что два пациента могут одновременно попытаться записаться на один и тот же слот. Чтобы этого избежать, используется следующий алгоритм:

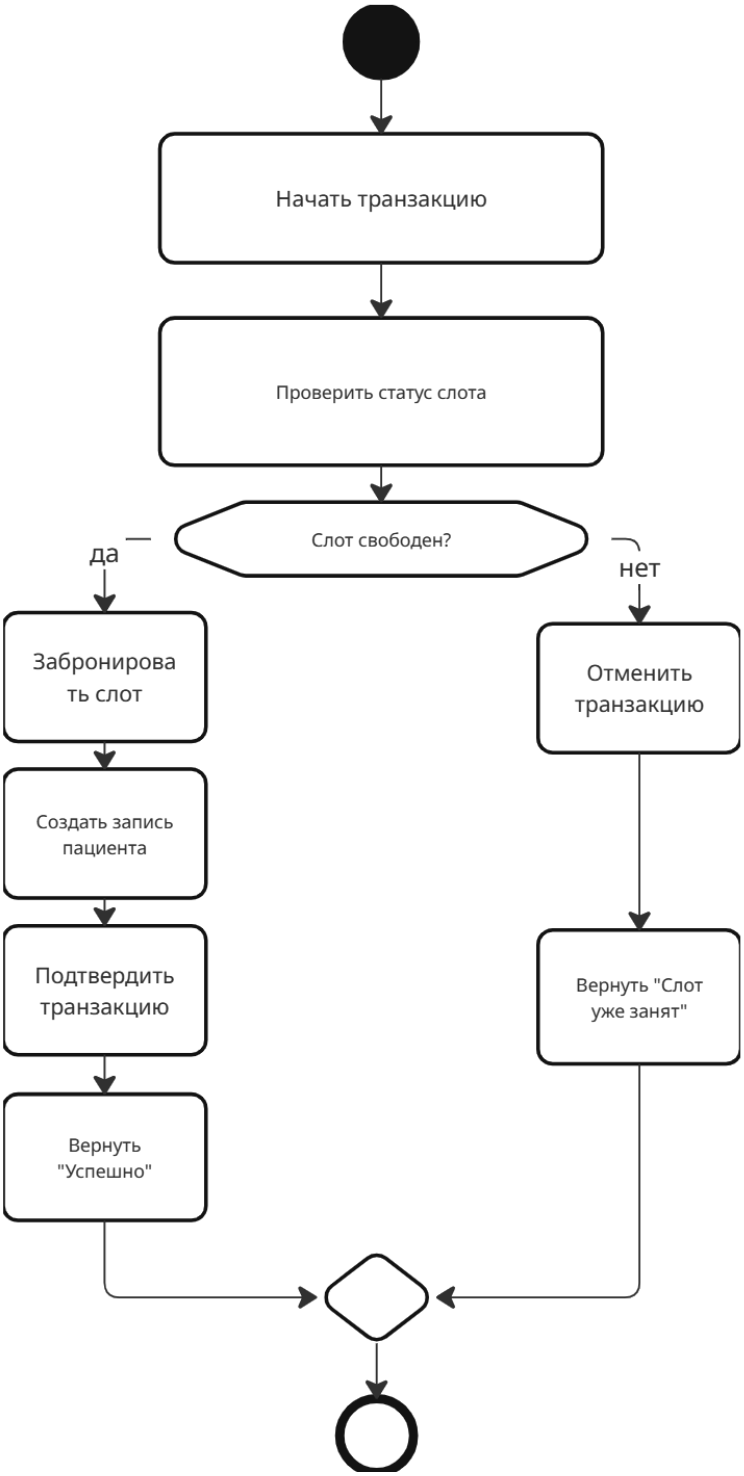

Рисунок 2.15 – Блок-схема алгоритма предотвращения двойной записи

Оценка сложности: Алгоритм выполняется за O(1) относительно количества данных — время зависит только от скорости работы базы данных.

Алгоритм отправки напоминаний

Чтобы пациенты не забывали о приёме, система отправляет напоминания. Алгоритм работает в фоновом режиме.

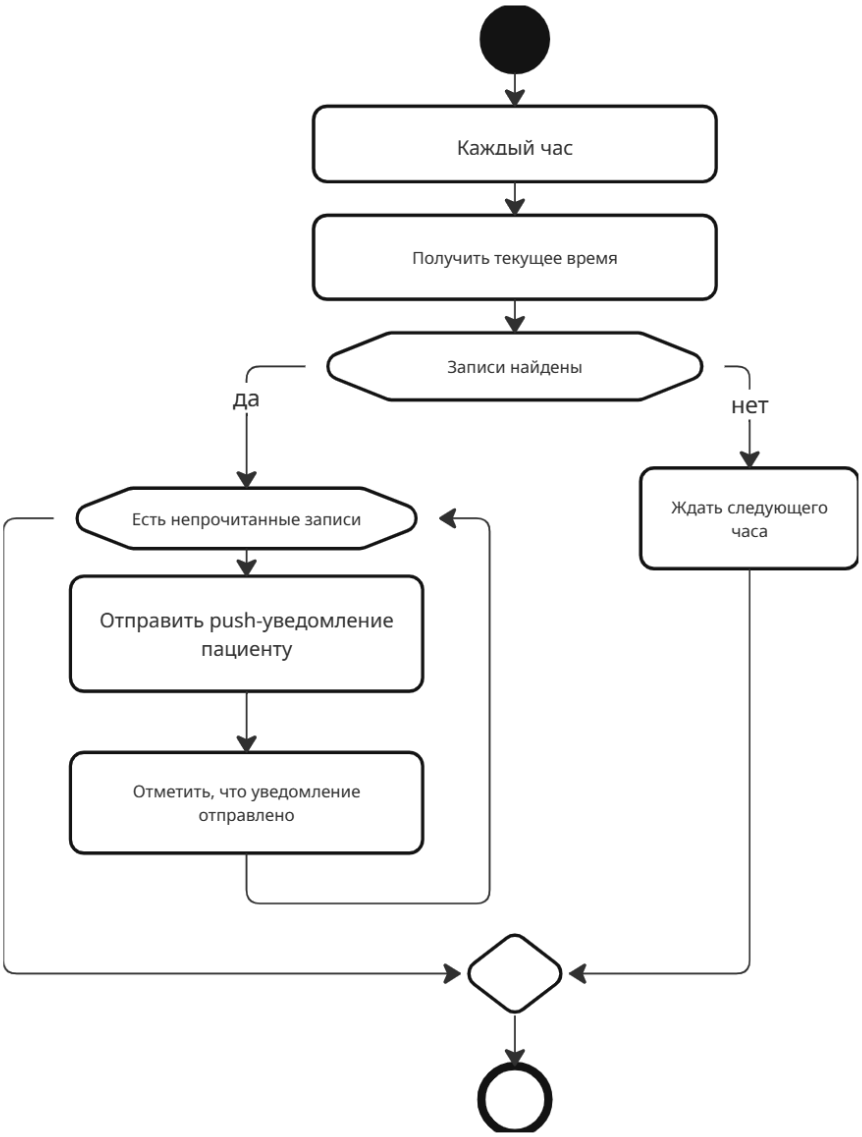

Рисунок 2.16 – Блок-схема алгоритма отправки напоминаний

Оценка сложности: Сложность составляет O(k), где k — количество записей на завтра. Это линейная зависимость — чем больше записей, тем дольше обработка, но для масштабов частной клиники это некритично.

Сводная таблица сложности алгоритмов

Для оценки эффективности разработанных алгоритмов был проведён анализ их вычислительной сложности. Под вычислительной сложностью понимается зависимость времени выполнения алгоритма от объёма входных данных. В таблице 2.1 представлены результаты оценки для ключевых алгоритмов системы.

Таблица 2.1 — Оценка сложности алгоритмов

Алгоритм	**Сложность**	**Пояснение**
Проверка доступности слота	O(log n)	Быстрый поиск по индексу
Поиск свободных слотов	O(m log m)	Сортировка отфильтрованных данных
Создание записи	O(1)	Простая вставка в БД
Предотвращение двойной записи	O(1)	Транзакция с блокировкой
Отправка напоминаний	O(k)	Обработка всех записей на завтра

Примечание: n — общее количество записей в системе; m — количество слотов у конкретного врача; k — количество записей, удовлетворяющих условию поиска.

Разработанные алгоритмы обеспечивают корректную работу всех основных функций системы: поиск свободного времени, запись на приём, предотвращение конфликтов при одновременной записи и отправку напоминаний. Оценка сложности показывает, что алгоритмы работают достаточно быстро даже при росте нагрузки, что соответствует требованиям технического задания.

2.5. Архитектура программного продукта

Архитектура программного продукта представляет собой структурное описание организации и взаимосвязей его компонентов или модулей, которое определяет общий план построения и функционирования программы. В данном разделе представлено описание архитектуры разрабатываемой системы самозаписи пациентов, обоснование выбранного подхода и графическое представление взаимодействия компонентов.

Выбор архитектурного решения

Для реализации системы самозаписи пациентов была выбрана клиент-серверная архитектура с выделением двух основных уровней:

- Клиентская часть (мобильное приложение) — отвечает за взаимодействие с пользователем, отображение данных и первичную валидацию вводимой информации.
- Серверная часть — обеспечивает хранение данных, реализацию бизнес-логики и обработку запросов от клиентов.

Выбор данной архитектуры обусловлен следующими преимуществами:

1. Разделение ответственности — клиент отвечает только за интерфейс, сервер — за данные и логику. Это упрощает разработку и поддержку системы.
2. Масштабируемость — при росте числа пользователей можно увеличить мощность сервера или добавить новые серверные узлы без изменения клиентского приложения.
3. Безопасность — все критичные данные хранятся на сервере, а доступ к ним контролируется через механизмы аутентификации и авторизации.
4. Кроссплатформенность — клиентская часть на Flutter работает на разных платформах, а серверная на Go может быть развернута в любой среде. 2.5.2

Компоненты архитектуры

Разрабатываемая система состоит из следующих основных компонентов:

1. Мобильный клиент (Flutter/Dart)
   - Отвечает за пользовательский интерфейс пациентов;
   - Реализует экраны: авторизация, поиск врачей, просмотр расписания, запись на приём, история посещений;
   - Выполняет локальное кэширование данных для работы при отсутствии соединения;
   - Отправляет HTTP-запросы к серверному API.
2. Сервер приложений (Go)
   - Реализует REST API для взаимодействия с клиентами;
   - Содержит бизнес-логику системы: проверка доступности, создание записей, управление расписанием;
   - Обеспечивает аутентификацию пользователей с помощью JWT-токенов;
   - Валидирует входящие данные перед записью в базу.
3. База данных (PostgreSQL)
   - Хранит всю информацию о пользователях, врачах, расписании и записях;
   - Обеспечивает целостность данных с помощью транзакций;
   - Поддерживает индексы для быстрого поиска.
4. Внешние сервисы (опционально)
   - Сервис отправки push-уведомлений (Firebase Cloud Messaging);
   - SMTP-сервер для email-уведомлений;
   - SMS-шлюз для отправки смс-напоминаний.

Схема взаимодействия компонентов

На рисунке 2.17 представлена схема архитектуры программного продукта, отражающая взаимодействие между основными компонентами системы.

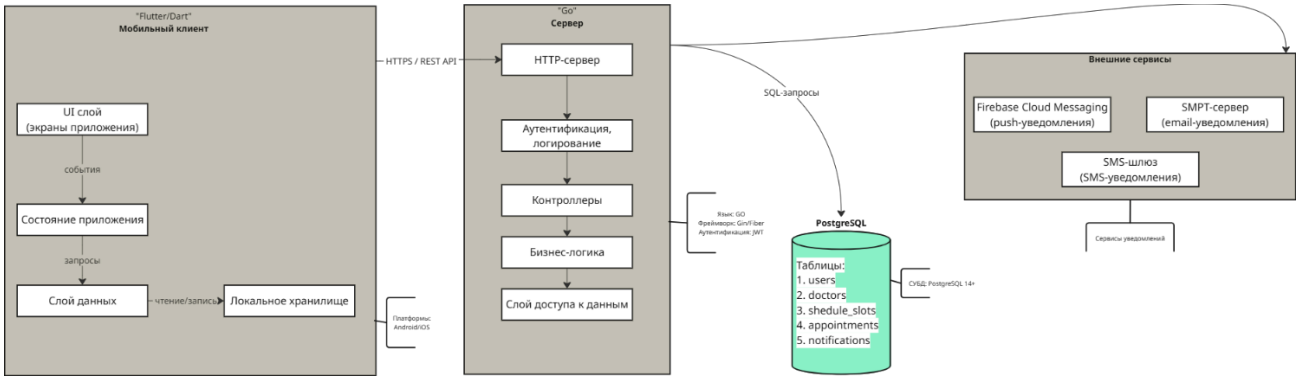

Рисунок 2.17 – Архитектура клиент-серверного приложения

Описание взаимодействия компонентов

Взаимодействие пользователя с системой:

1. Пользователь открывает мобильное приложение и проходит аутентификацию.
2. Клиент отправляет запрос на сервер с учётными данными.
3. Сервер проверяет данные и возвращает JWT-токен для последующих запросов.
4. При каждом следующем действии клиент включает токен в заголовки запросов.

Сценарий записи на приём:

1. Пользователь выбирает врача и желаемое время.
2. Клиент отправляет POST-запрос на /api/appointments с данными записи.
3. Сервер получает запрос, проверяет валидность токена.

4. Бизнес-логика проверяет доступность выбранного слота (алгоритм из раздела 2.4.2).
5. Если слот свободен, сервер в транзакции создаёт запись и обновляет статус слота.
6. При успехе сервер возвращает подтверждение клиенту.
7. Параллельно сервер инициирует отправку уведомления через внешние сервисы.

Обработка данных:

- Все запросы к базе данных выполняются через слой доступа к данным с использованием параметризованных запросов для защиты от SQL-инъекций.
- Для оптимизации производительности часто запрашиваемые данные (например, список врачей) могут кэшироваться на стороне клиента.

Разработанная архитектура клиент-серверного приложения обеспечивает:

- Чёткое разделение ответственности между компонентами;
- Масштабируемость при росте числа пользователей;
- Безопасность хранения и передачи данных;
- Возможность независимой модификации клиентской и серверной частей;
- Соответствие всем функциональным и нефункциональным требованиям, сформулированным в техническом задании (Приложение А).

Выбранный стек технологий (Flutter + Go + PostgreSQL) является современным, хорошо документированным и позволяет эффективно реализовать все заявленные функции системы самозаписи пациентов.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Багин С. Послали на сайт. Почему так сложно записаться к врачу? [Электронный ресурс] / С. Багин // Аргументы и Факты. — 2024. — 28 августа. — Режим доступа: https://udm.aif.ru/health/situation/poslali-na-sayt-pochemu-tak-slozhno-zapisatsya-k-vrachu (дата обращения: 19.03.2026).
2. Каждый второй россиянин выбирает частную медицину из-за возможности быстро попасть на прием любого специалиста [Электронный ресурс] // Remedium.ru. — 2025. — 28 ноября. — Режим доступа: https://remedium.ru/news/kazhdyy-vtoroy-rossiyanin-vybi/ (дата обращения: 19.03.2026).
3. Мезенцева Т.А. Оценка удовлетворенности населения организацией записи на прием к врачу в медицинских организациях Свердловской области [Электронный ресурс] / Т.А. Мезенцева, Е.В. Ютяева, С.Л. Леонтьев [и др.] // Современные проблемы здравоохранения и медицинской статистики. — 2024. — № 2. — С. 707-726. — Режим доступа: https://healthproblem.ru/files/pdf/1350-pdf.pdf (дата обращения: 19.03.2026).
4. Пользователи портала госуслуг чаще всего жалуются на сервис записи к врачу [Электронный ресурс] // Vademec.ru. — 2023. — 3 мая. — Режим доступа: https://vademec.ru/news/2023/05/03/polzovateli-portala-gosuslug-chashche-vsego-zhaluyutsya-na-servis-zapisi-k-vrachu/ (дата обращения: 19.03.2026).
5. Россияне теряют недели в ожидании приема у нужного врача [Электронный ресурс] // Grand-clinic.ru. — 2022. — 29 ноября. — Режим доступа: https://www.grand-clinic.ru/media/publikatsii-v-presse/rossiyane-teryayut-nedeli-v-ozhidanii-priema-u-nuzhnogo-vracha-versiya-dlya-pechati-obsudit-na-forum/ (дата обращения: 19.03.2026).

6. Филь Т.С. Цифровое здравоохранение Российской Федерации: основные понятия для практикующих врачей / Т.С. Филь, И.Г. Бакулин. — Москва: ГЭОТАР-Медиа, 2022. — 128 с. (дата обращения: 19.03.2026).
7. ГОСТ 34.602–2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. — Москва : Стандартинформ, 2020. — 18 с.
8. Книги и статьи на русском языке Берсенева Е. А. Создание мобильного приложения врача для осуществления дистанционного мониторинга / Е. А. Берсенева, И. М. Корсаков, А. Г. Михайлова // Проблемы социальной гигиены, здравоохранения и истории медицины. — 2015. — № 5. — С. 45–49.
9. Дикинштейн М. В. Разработка мобильного приложения для системы домашнего мониторинга ЭКГ : выпускная квалификационная работа / М. В. Дикинштейн, Д. С. Лобунцов. — Тюмень : ТюмГУ, 2018. — 54 с.
10. Казакова А. А. Разработка автоматизированной системы и мобильного приложения для обслуживания пациентов в поликлинике : ВКР / А. А. Казакова. — Санкт-Петербург : СПбГУ, 2020. — 69 с.
11. Мезенцева Т. А. Оценка удовлетворенности населения организацией записи на прием к врачу в медицинских организациях Свердловской области [Электронный ресурс] / Т. А. Мезенцева, Е. В. Ютяева, С. Л. Леонтьев [и др.] // Современные проблемы здравоохранения и медицинской статистики. — 2024. — № 2. — С. 707–726. — URL: https://healthproblem.ru/files/pdf/1350-pdf.pdf (дата обращения: 19.03.2026).
12. Тивари А. Профессиональный Go / А. Тивари. — Москва : Эксмо, 2023. — 480 с.
13. Уиндлз М. Флаттер в действии / М. Уиндлз. — Москва : ДМК Пресс, 2021. — 368 с.

14. Филь Т. С. Цифровое здравоохранение Российской Федерации: основные понятия для практикующих врачей / Т. С. Филь, И. Г. Бакулин. — Москва : ГЭОТАР-Медиа, 2022. — 128 с.

ПРИЛОЖЕНИЯ

Приложение А. Техническое задание на разработку

Приложение А. Техническое задание на разработку

1 Введение

1.1 Наименование программы

Наименование программы – «Система самозаписи пациентов частной клиники» («MedBook»).

1.2 Краткая характеристика области применения

Программная система «MedBook» предназначена для автоматизации процесса записи пациентов на приём к врачам в частной клинике. Система обеспечивает:

- самозапись пациентов через мобильное приложение;
- централизованное управление расписанием врачей;
- отображение актуальной информации о доступных временных интервалах;
- уведомление пациентов о предстоящих приёмах и изменениях в расписании.

Система применяется в частных медицинских организациях, оказывающих амбулаторные и консультативные услуги, и ориентирована на упрощение взаимодействия пациента с клиникой, снижение нагрузки на регистратуру и повышение качества сервиса.

2 Основания для разработки

Основанием для разработки является задание на выпускную квалификационную работу и необходимость создания системы самозаписи пациентов для частной клиники.

Техническое задание разработано в соответствии с требованиями:

– ГОСТ 34.602–2020 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы»;

3 Назначение разработки

Система будет использоваться следующими группами пользователей:

– **пациенты** – самозапись на приём к врачу, просмотр и управление своими записями, получение уведомлений;
– **администраторы клиники** – управление расписанием и записями, просмотр отчётности;
– **врачи** (опционально) – просмотр собственного расписания и списка пациентов на приём.

3.1 Функциональное назначение

Для пациентов система должна обеспечивать:

– регистрацию и аутентификацию пользователя;
– просмотр списка врачей и медицинских услуг;
– поиск врачей и услуг с использованием фильтров (специализация, дата, время и др.);
– просмотр доступных временных интервалов для приёма;
– создание записи на приём;
– просмотр списка своих записей;
– перенос и отмену записей с учётом регламентов клиники;
– получение уведомлений о создании, переносе и отмене записей.

Для администраторов система должна обеспечивать:

– управление расписанием врачей (создание, изменение, закрытие временных слотов);
– просмотр списка записей пациентов;

– принудительное создание, перенос и отмену записей;
– формирование отчётов по загрузке врачей и обращаемости пациентов;
– управление учётными записями пользователей (при необходимости).

Для врачей (если предусмотрена соответствующая роль):

– просмотр личного расписания;
– просмотр списка пациентов, записанных на приём.

3.2 Эксплуатационное назначение

Система эксплуатируется в виде клиент–серверного приложения.

– **Клиентская часть** реализуется в виде мобильного приложения на базе фреймворка Flutter и предназначена для работы на мобильных устройствах под управлением Android и iOS.
– **Серверная часть** реализуется на языке Go, разворачивается на облачной или локальной серверной инфраструктуре и обеспечивает обработку запросов клиентов, доступ к базе данных и реализацию бизнес-логики.

4 Требования к программе

4.1 Требования к функциональным характеристикам

4.1.1 Требования к составу выполняемых функций

Система должна обеспечивать выполнение следующих основных функций:

Управление пользователями:

– регистрация пациента;
– аутентификация и выход из системы;
– восстановление доступа (через email/телефон – при необходимости);
– просмотр и редактирование профиля пользователя (контактные данные, дата рождения и др.).

Работа с расписанием и записями (для пациентов):

– просмотр списка врачей и услуг;
– поиск врачей по специализации, ФИО, отделению;
– просмотр доступных временных интервалов для выбранного врача/услуги;
– создание записи на приём;
– просмотр списка собственных активных и прошедших записей;
– перенос записи (изменение даты/времени приёма) с учётом ограничений;
– отмена записи с учётом регламентов клиники.

Работа с расписанием и записями (для администраторов):

– просмотр общего расписания клиники и расписаний отдельных врачей;
– создание записи от имени пациента (по телефону/личному обращению);
– изменение и отмена записей пациентов;
– закрытие и открытие временных интервалов (например, на время отпуска врача);
– управление базой врачей и услуг (создание, редактирование, деактивация).

Отчётность и аналитика (для администраторов):

– формирование отчётов по загрузке врачей;
– отчёты по числу записей, отмен, неявок (при наличии данных);
– выгрузка отчётных данных в табличный формат (при необходимости).

Уведомления:

– отправка уведомления пациенту о создании записи;
– отправка напоминания о предстоящем приёме;
– уведомления об изменении или отмене записи.

4.1.2 Требования к организации входных и выходных данных

Входными данными системы являются:

– регистрационные данные пользователя (ФИО, контактный телефон, email, пароль и др.);
– данные для аутентификации (логин/телефон/email, пароль);

– параметры поиска (специализация врача, дата, время, тип услуги и др.);
– данные записи на приём (выбранный врач, дата и время приёма);
– запросы на изменение/отмену записи;
– данные для формирования расписания (рабочие дни, время приёма, длительность слотов).

Выходные данные:

– список врачей и медицинских услуг;
– расписание с доступными временными интервалами;
– подтверждённая запись на приём;
– изменённая или отменённая запись;
– список активных и прошедших записей пациента;
– актуальное расписание для административного персонала;
– отчёты по загрузке врачей и обращениям пациентов;
– уведомления, отображаемые на устройстве пользователя (push, внутренняя лента уведомлений).

4.1.3 Требования к временным характеристикам

В процессе тестирования проверялось соответствие системы следующим требованиям к временным характеристикам:

– время отклика пользовательского интерфейса на действия пользователя (навигация, переключение экранов, локальные операции) – не более 1 секунды;
– не менее 95 % API-запросов к серверу должны обрабатываться менее чем за 500 мс при номинальной нагрузке;
– время первичного запуска мобильного приложения (cold start) – не более 3 секунд на минимально поддерживаемых устройствах.

4.2 Требования к надёжности

Надёжность системы должна удовлетворять следующим требованиям.

4.2.1 Общие требования к надёжности

- вероятность безотказной работы системы в расчёте на месяц эксплуатации – не менее 99 % при корректной эксплуатации;
- время восстановления после сбоя серверной части – не более 10 минут при наличии резервной инфраструктуры.

4.2.2 Обеспечение устойчивого функционирования

Для обеспечения устойчивого функционирования система должна:

- реализация механизма повтора запросов при временных сетевых сбоях;
- обработка ошибок сети с уведомлением пользователя и попыткой автоматического восстановления;
- серверная часть должна обеспечивать резервное копирование базы данных не реже одного раза в сутки;
- при недоступности сервера мобильное приложение должно корректно информировать пользователя и не допускать неконсистентных операций.

4.2.3 Восстановление после отказа

Восстановление после отказа должно обеспечивать:

- автоматическое восстановление сетевого соединения при возобновлении доступа к сети;
- сохранение состояния пользовательского интерфейса при сворачивании/неожиданном закрытии приложения (по возможности);
- восстановление работоспособности после перезапуска сервера без потери данных, сохранённых в базе.

4.3 Условия эксплуатации

Условия эксплуатации системы определяются следующими характеристиками.

4.3.1 Клиентские устройства и платформы

Поддерживаемые мобильные операционные системы:

- Android 10 (API 29) и выше;
- iOS 14.0 и выше.

(Опционально, при расширении проекта возможно добавление веб-клиента.)

4.3.2 Минимальные требования к клиентским устройствам

Мобильные устройства:

- процессор: 4-ядерный, от 1.8 ГГц;
- оперативная память: не менее 2 ГБ;
- свободное место для установки: не менее 100 МБ;
- экран: разрешение не ниже 1280×720.

4.3.3 Серверная инфраструктура

Минимальные требования к серверу (для тестовой/пилотной эксплуатации):

- процессор: 2–4 vCPU;
- оперативная память: 4–8 ГБ;
- хранилище: SSD объёмом не менее 50–100 ГБ;
- операционная система: Ubuntu Server 22.04 LTS или аналогичная;
- СУБД: PostgreSQL 14+;
- доступ к сети Интернет с пропускной способностью не ниже 100 Мбит/с.

В дальнейшем допускается масштабирование инфраструктуры в зависимости от количества пользователей и нагрузки.

4.3.4 Климатические условия эксплуатации

Специальные климатические условия не требуются, достаточно соблюдения стандартных требований к эксплуатации вычислительной техники, указанных производителями оборудования.

4.4 Требования к информационной и программной совместимости

Требования к информационной и программной совместимости включают:

4.4.1 Клиентская часть

Клиентская часть должна соответствовать следующим требованиям:

- фреймворк: Flutter 3.x и выше;
- язык программирования: Dart;
- поддержка платформ: Android, iOS;
- рекомендуется использование архитектурных подходов (например, BLoC/Cubit или аналогичных) для разделения логики и представления;
- поддержка локального кэширования данных (например, с использованием встроенных механизмов хранения Flutter).

Конкретный набор библиотек и пакетов выбирается разработчиком с учётом целей ВКР.

4.4.2 Серверная часть

Серверная часть должна соответствовать следующим требованиям:

- язык программирования: Go (актуальная стабильная версия на момент разработки, не ниже 1.20);
- использование стандартной библиотеки net/http или фреймворка на усмотрение разработчика (например, Gin/Fiber/chi);
- база данных: PostgreSQL 14+;

– механизм аутентификации: JWT-токены или аналогичный, обеспечивающий stateless-аутентификацию;
– доступ к базе данных через драйвер/ORM, поддерживающий PostgreSQL.

4.4.3 Протоколы обмена

Протоколы обмена данными должны удовлетворять следующим требованиям:

– основной протокол взаимодействия клиент–сервер: REST API поверх HTTPS;
– формат обмена данными: JSON;
– шифрование: TLS версии не ниже 1.2;
– аутентификация: использование токенов доступа (JWT) в заголовках запросов.

4.5 Специальные требования

Специальные требования к системе включают:

4.5.1 Безопасность

Требования к безопасности:

– все сетевые запросы между клиентом и сервером должны выполняться по защищённому протоколу HTTPS;
– пароли пользователей хранятся только в виде криптографических хэш-значений с использованием современного алгоритма (например, bcrypt/argon2);
– система должна быть защищена от основных уязвимостей веб-приложений (SQL-инъекции, XSS, атаки типа «подбор пароля» и др.) за счёт валидации входных данных, использования параметризованных запросов и механизмов ограничения частоты обращений;

– доступ к административному интерфейсу должен осуществляться только по защищённым каналам с обязательной аутентификацией.

4.5.2 Производительность

Требования к производительности:

– 95 % запросов к API при номинальной нагрузке должны выполняться за время не более 500 мс;
– система должна поддерживать одновременную работу не менее 100 активных пользователей (для прототипа) с возможностью масштабирования до большего числа при расширении проекта.

4.5.3 Масштабируемость

Требования к масштабируемости:

– архитектура серверной части должна предусматривать возможность горизонтального масштабирования (запуск нескольких экземпляров сервиса за балансировщиком);
– структура базы данных должна быть спроектирована с учётом использования индексов по часто используемым полям (пациент, врач, дата/время приёма);
– рекомендуется использование кэширования часто запрашиваемых данных (например, расписаний).

4.5.4 Удобство использования

Требования к удобству использования:

– пользовательский интерфейс должен корректно отображаться на экранах с различным разрешением и плотностью пикселей;

– количество действий, необходимых для оформления новой записи на приём, должно быть минимальным (по возможности не более 3–4 основных шагов);
– в интерфейсе должны быть предусмотрены подсказки и сообщения об ошибках, понятные пользователю.

4.6 Требования к маркировке и упаковке

Требования к маркировке и упаковке программного продукта:

Программный продукт распространяется в виде мобильного приложения через официальные магазины (Google Play, App Store) либо в виде установочных пакетов для тестовой эксплуатации. Специальных требований к маркировке и упаковке не предъявляется.

4.7 Требования к транспортированию и хранению

Требования к транспортированию и хранению компонентов системы:

Цифровые компоненты системы (исходный код, исполняемые файлы, базы данных, резервные копии) хранятся в электронном виде на носителях и серверах, обеспечивающих сохранность и конфиденциальность данных. Специальных требований к транспортированию и хранению не предъявляется, за исключением соблюдения требований информационной безопасности.

5 Требования к программной документации

Состав программной документации определяется ГОСТ 34.602–2020 и ГОСТ 19.201–78 и включает, как минимум:

– Техническое задание;
– Технический проект (при необходимости);
– Программу и методику испытаний (ПМИ);
– Руководство пользователя (пациента);

– Руководство администратора системы;
– Описание программного продукта (структура, архитектура, интерфейсы);
– Пояснительную записку к выпускной квалификационной работе.

По усмотрению разработчика и в соответствии с требованиями ВКР могут быть оформлены дополнительные документы: спецификация программы, формуляр, ведомость эксплуатационных документов и др.

6 Технико-экономические показатели

Внедрение системы «МедЗапись» в частной клинике позволяет:

– снизить нагрузку на регистратуру за счёт переноса части операций записи на самих пациентов;
– уменьшить количество ошибок и накладок в расписании;
– повысить удовлетворённость пациентов за счёт удобного и прозрачного процесса записи;
– ускорить процесс планирования и анализа загрузки врачей.

Экономический эффект достигается за счёт:

– сокращения времени, затрачиваемого персоналом регистратуры на оформление записей;
– повышения количества успешно состоявшихся приёмов благодаря напоминаниям и удобству переноса;
– улучшения управления расписанием и оптимизации использования рабочего времени врачей.

7 Стадии и этапы разработки

Разработка программного продукта выполняется в следующие стадии:

7.1 Стадия 1: Техническое задание

На данной стадии выполняются следующие работы:

– исследование предметной области и существующих решений;
– формулирование и анализ требований к системе;
– разработка и согласование технического задания.

7.2 Стадия 2: Технический проект и рабочая документация

На данной стадии выполняются следующие работы:

– проектирование архитектуры клиент–серверного приложения;
– проектирование структуры базы данных;
– разработка прототипа мобильного клиента (Flutter);
– разработка серверной части (Go, PostgreSQL);
– подготовка программной документации;
– проведение модульного и интеграционного тестирования.

7.3 Стадия 3: Внедрение и испытания

На данной стадии выполняются следующие работы:

– развёртывание системы в тестовой среде;
– проведение приёмочных испытаний согласно ПМИ;
– анализ результатов испытаний и исправление выявленных замечаний;
– подготовка рекомендаций по эксплуатации и дальнейшему развитию системы.

8 Порядок контроля и приёмки

Порядок контроля и приёмки системы включает следующие положения.

8.1 Виды испытаний

Проводятся следующие виды испытаний:

- функциональное тестирование (проверка реализации всех заявленных функций);
- интеграционное тестирование (взаимодействие клиента, сервера и базы данных);
- нагрузочное тестирование (проверка работы при заданном числе одновременных пользователей);
- тестирование безопасности (проверка защиты аутентификации и обработки данных);
- тестирование удобства использования (оценка понятности интерфейса для пациентов и администраторов).

8.2 Критерии приёмки

Система считается прошедшей приёмочные испытания при выполнении следующих условий:

- реализованы и корректно функционируют все ключевые функции, указанные в разделе 4.1;
- соблюдены основные требования к временным характеристикам и производительности;
- отсутствуют критические дефекты, приводящие к потере данных или невозможности выполнения основных операций (регистрация, авторизация, создание записи);
- корректно обеспечивается аутентификация и разграничение прав доступа;
- документация оформлена и соответствует фактическому состоянию программного продукта.

8.3 Процедура приёмки

Приемка программного продукта осуществляется по следующему алгоритму:

- испытания проводятся в соответствии с утверждённой Программой и методикой испытаний;
- результаты испытаний фиксируются в протоколе, в котором указываются выявленные замечания и решения по их устранению;
- по итогам успешных испытаний составляется акт приёмки-сдачи программного продукта (для ВКР – соответствующий раздел отчётной документации и заключение руководителя/комиссии).